Parakh Portal - CI/CD Process & Architecture Specification

Inspection & Governance Portal | MPSEDC, Govt. of Madhya Pradesh

CMMI-DEV Level 3 | Doc: PRKH-CICD-001

1. System Repositories & Architecture Overview

CMMI Process Area: CM (Configuration Management)

In accordance with **CMMI Level 3 CM SP 1.1 (Identify Configuration Items)** and **CM SP 1.2 (Establish a Configuration Management System)**, all Parakh source code, database migration assets, mobile applications, and infrastructure-as-code deployment definitions are managed in dedicated repositories under the **MPSEDC-ORG** Azure DevOps tenant:

Table 1.1: Core Configuration Items (CIs) & Repositories

CI Identifier	Repository Name	Technology Stack	Primary Branches	Access Policy	Remote Git URL
CI-FE-01	**Parakh** (Frontend)	Angular 14+, TypeScript, Node.js, Leaflet GIS	`develop`, `staging`, `main`	Branch Policy / Mandatory PR	`https://dev.azure.com/MPSEDC-ORG/Parakh/_git/Parakh`
CI-BE-02	**ParakhAPI** (Backend)	ASP.NET Core 6.0 Web API, C#, PostgreSQL, eSign SDK	`develop`, `staging`, `main`	Branch Policy / Mandatory PR	`https://dev.azure.com/MPSEDC-ORG/Parakh/_git/ParakhAPI`
CI-DB-03	**parakh_db** (Database)	PostgreSQL Schema DDL/DML, Migration Scripts	`develop`, `main`	DB Admin / Peer Review	`https://dev.azure.com/MPSEDC-ORG/Parakh/_git/parakh_db`
CI-MB-04	**ParakhMobile** (Mobile)	Flutter / Android Field Inspection App	`develop`, `main`	Branch Policy / Mandatory PR	`https://dev.azure.com/MPSEDC-ORG/Parakh/_git/ParakhMobile`

Table 1.2: Role-Based Authorization & Change Matrix (CMMI CM SP 1.2 / SP 2.2)

Role / Stakeholder	Assigned Personnel	Git Repository Permissions	Pipeline Trigger Permissions	Environment Deployment Authority
DevOps / Module Lead	Aman Mishra	Create feature branches, submit & approve PRs, manage pipeline YAMLs	Trigger & manage CI/CD pipelines across all pools	Deploy to Development, Staging (UAT) & Production
Technical Lead	Madhur Nema	Repository Administrator, mandatory PR code reviewer	Trigger CI/CD pipelines, configure agent pools	Review & endorse Production deployment baselines

Role / Stakeholder	Assigned Personnel	Git Repository Permissions	Pipeline Trigger Permissions	Environment Deployment Authority
Lead Business Analyst	Dheeraj	Read access to repos & work items, sprint backlog manager	View build execution & deployment status	UAT acceptance sign-off & functional release concurrence
QA Lead	Manisha	Read access to repos, manage test suites in Azure Test Plans	View deployment status on Staging	Sign-off test execution reports (142 Test Cases)
Project Manager / CAB	Ajeet	Project Administrator, final release gatekeeper	Approve production pipeline releases	Final Go-Live & CAB approval for production releases

2. Branching Strategy & Code Promotion Lifecycle

CMMI Process Area: VER (Verification) & CM

The Parakh development lifecycle enforces a strict promotion hierarchy. Unreviewed, untested, or unvalidated code is prevented from reaching testing or production environments through automated gates:

VISUAL FLOW: GATED CODE PROMOTION ARCHITECTURE

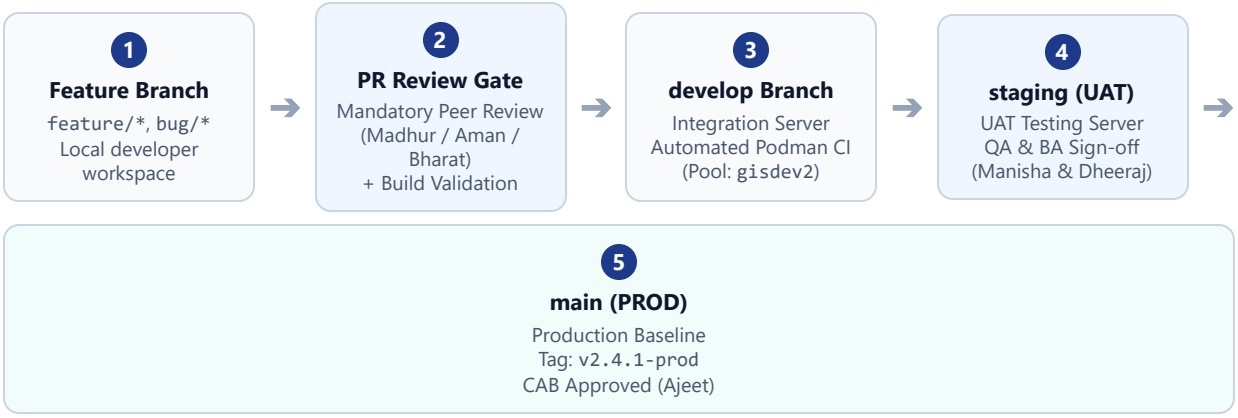


Table 2.1: Code Promotion Gates & CMMI Verification Criteria (CMMI VER SP 1.1 / VAL SP 1.1)

Transition Stage	Source & Target	Mandatory Verification Gate	Approver / Authority	Objective Audit Evidence
Gate 1: Code Integration	`feature/*` → `develop`	1 mandatory peer code review, clean compile, no merge conflict, linked Azure Boards Work Item.	Madhur Nema / Aman Mishra	Completed Pull Request (e.g. PR #14729, PR #14721) with approver timestamp.
Gate 2: UAT Promotion	`develop` → `staging`	Automated Podman build success, SonarQube quality gate passed, regression test suite executed.	Aman Mishra / Madhur Nema	CI Build Run (Build #21094), CD deployment log on Staging server.

Transition Stage	Source & Target	Mandatory Verification Gate	Approver / Authority	Objective Audit Evidence
Gate 3: Production Rollout	`staging` ➔ `main`	142 functional test cases passed, formal UAT acceptance sign-off, Change Advisory Board (CAB) approval.	Dheeraj (BA) & Ajeet (PM)	Semantic Git Tag (e.g. `v2.4.1-prod`), Release Note, UAT Sign-off memo.

3. CI/CD Pipelines & Podman Containerized Builds

CMMI Process Area: PI (Product Integration)

In accordance with **CMMI PI SP 1.1 (Determine Integration Sequence)** and **PI SP 2.1 (Assemble Product Components)**, builds run inside isolated **Podman containers** on dedicated self-hosted agent pools (`gisdev2` / `gisstag2`) to ensure 100% reproducible builds free of host contamination:

Table 3.1: Azure DevOps Pipeline Engineering Matrix

Pipeline Name	ID	Component	Agent Pool	Podman Container Runtime	Execution Commands & Steps	Artifact Staging Path
`CI_Parakh`	**676**	Frontend (Angular)	`gisdev2` / `gisstag2`	`node:18-alpine`	`npm ci` (with automated fallback to `npm install`), `npx ng build --configuration={env} --base-href ./`	`$(CIBuildRoot)/Parakh/CI_Parakh`
`CD_Parakh`	**731**	Frontend (Nginx)	`gisdev2` / `gisstag2`	Native Shell / Rsync	Pre-deploy snapshot backup, `rsync -a --delete` to web root, `chmod 755`, `systemctl reload nginx`	`/var/www/html/parakh/`
`CI_Parakh_API`	**504**	Backend (.NET 6)	`gisdev2`	`mcr.microsoft.com/dotnet/sdk:6.0`	`dotnet restore`, `dotnet build -c Release --no-restore`, `dotnet publish -c Release --no-build`	`$(CIBuildRoot)/ParakhAPI/CI_Parakh_API`
`CD_Parakh_API`	**732**	Backend (Service)	`gisdev2`	Native Shell / systemd	Binary snapshot backup, atomic rsync, service restart (`systemctl restart parakhapi.service`), HTTP 200 health verification	`/var/www/api/parakhapi/`

VISUAL FLOW: CONTAINERIZED BUILD & PACKAGING LIFECYCLE

1. Checkout Source
Azure Repos Git checkout (Clean workspace)
→
2. Podman Container
Isolated runtime (Node.js 18 / .NET 6 SDK)
→
3. Build & Compile
Dependency restore, production optimization
→
4. Publish Artifact
Versioned drop folder ready for CD deployment

4. Deployment Safety: Backup & Automated Rollback

CMMI Process Area: RSKM (Risk Management) & PMC

Deployment scripts incorporate automated fault tolerance to guarantee high availability. A complete snapshot backup is taken prior to modifying application files, and an automated rollback is triggered within 3 minutes if any error occurs:

VISUAL ARCHITECTURE: DEPLOYMENT & AUTOMATED RECOVERY FORK

Step 1: Backup
Atomic snapshot taken to `/backup/preserve_{BuildId}/`
→
Step 2: Deploy
Atomic rsync mirror + preserve SSL and config files
→
Step 3: Verification
Service restart + HTTP health probe (`/api/health`)

✅ **SCENARIO A: DEPLOYMENT SUCCESS (HTTP 200 OK)**

- Rsync mirror completed without error.
- Nginx / Kestrel services restarted successfully.
- Automated smoke tests confirm live application health.
- Deployment logged in Release Register; notification sent to team.

🚨 **SCENARIO B: AUTOMATED ROLLBACK TRIGGERED**

- Detected error during file sync, permissions, or health check.
- **Automated Rollback:** Pipeline restores `/backup/` snapshot.
- Web service reloaded on previous working baseline (< 3 mins).
- Build marked as failed; automated alert dispatched to DevOps lead.

Table 4.1: Deployment Failure Controls & Disaster Recovery Specifications

Failure Scenario	Detection Trigger	Automated Recovery Action	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Partial File Mirror / Network Interruption	Non-zero exit code during `rsync -a --delete`	Halt pipeline immediately, copy `/backup/preserve_{BuildId}/` back to target root.	< 2 minutes	0 seconds (Previous state restored)
Configuration / Syntax Error in Nginx	`nginx -t` test failure or systemctl reload error	Restore previous Nginx directives from snapshot, execute service reload.	< 1 minute	0 seconds

Failure Scenario	Detection Trigger	Automated Recovery Action	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Backend Service Health Check Failure	HTTP 500 / Timeout on `/api/health` probe	Revert published binaries from backup snapshot, restart `parakhapi.service`.	< 3 minutes	0 seconds
Corrupted Container Build Dependency	Checksum failure during `npm ci`	Automated fallback to `npm install`; if compile fails, deployment step is aborted.	Zero downtime (No files deployed)	0 seconds